

Bases de Datos

Semana 3 · S3 — Lab Libre | Auditoría DDL · Restricciones · FK semánticas · Pruebas de integridad
Grupos de 3–4 personas · Base de datos: su proyecto

#	Nombre completo	Código	Firma
1	Miguel Andres Contreras Rodriguez	20232020020	
2	Santiago Andrés Benavides Coral	20232020036	
3	Sergio Nicolas Osorio Guevara	20241020073	
4	Adiel Valentin Hernandez Manosalva	20201020144	

Entrega de hoy (S3)

Un único PDF por grupo subido al aula virtual. El PDF debe incluir:

1. El script DDL revisado completo (CREATE TABLE con todas las restricciones).
2. Los ALTER TABLE que aplicaron sobre su BD existente.
3. Screenshots de las pruebas de integridad: al menos 1 violación exitosa por tipo de restricción.

Entregable de proyecto de esta semana: DDL revisado con restricciones completas y FK semánticas.

¿Qué hacen hoy?

Tienen el DDL de su proyecto desde las semanas 1 y 2. Hoy lo formalizan con restricciones completas, ajustan el comportamiento de cada FK con lógica de negocio, y prueban que todo funcione intentando violarlo.

PARTE 1 Auditoría del DDL — tabla de chequeo por campo.

PARTE 2 Revisión de FK — ON DELETE con justificación de negocio.

PARTE 3 Pruebas de integridad — violar restricciones a propósito y leer el error.

PARTE 1 Auditoría del DDL

30 min — Revisen campo por campo qué restricciones tienen y cuáles faltan

Instrucciones:

1. Para cada tabla del proyecto, completen una fila por campo en la tabla de auditoría.
2. En las columnas NOT NULL, CHECK y DEFAULT: marquen ✓ si ya lo tienen, X si falta, o — si no aplica.
3. En la columna FK / ON DELETE: escriban RESTRICT, CASCADE o SET NULL según el comportamiento actual.

4. Después de completar la tabla, ejecuten los ALTER TABLE necesarios para agregar lo que falta.

Tabla 1 — Nombre: Client

Tabla	Campo	Tipo	NOT NULL	CHECK	DEFAULT	FK / ON DELETE
client	id_cliente	SERIAL	✓	—	—	PK — sin FK
client	first_name	VARCHAR(80)	✓	—	—	—
client	last_name	VARCHAR(80)	✓	—	—	—
client	email	VARCHAR(120)	✓	✓	—	—
client	phone	VARCHAR(20)	✓	✗	—	—
client	password_hash	VARCHAR(255)	✓	—	—	—
client	address	VARCHAR(200)	✗	—	—	—

ALTER TABLE para agregar restricciones faltantes (Tabla 1):

```
ALTER TABLE clients
ALTER COLUMN phone SET NOT NULL,
ADD CONSTRAINT chk_email CHECK (email LIKE '%@%');
```

```
143 ✓ ALTER TABLE clients
144     ALTER COLUMN phone SET NOT NULL,
145     ADD CONSTRAINT chk_email CHECK (email LIKE '%@%');
146
147
148
```

Data Output Messages Notifications

ALTER TABLE

Query returned successfully in 33 msec.

Tabla 2 — Nombre: Booking

Tabla	Campo	Tipo	NOT NULL	CHECK	DEFAULT	FK / ON DELETE
booking	id_booking	SERIAL	✓	—	—	PK — sin FK

booking	id_cliente	INT	✓	—	—	FK Client — RESTRICT
booking	total_price	DECIMAL(10,2)	✓	✓	—	—
booking	datetime_start	TIMESTAMP	✓	—	—	—
booking	datetime_end	TIMESTAMP	✓	✓	—	—
booking	status	VARCHAR(20)	✓	✓	✓	—

ALTER TABLE para agregar restricciones faltantes (Tabla 2):

```
ALTER TABLE bookings
  DROP CONSTRAINT bookings_id_cliente_fkey,
  ADD CONSTRAINT bookings_id_cliente_fkey FOREIGN KEY (id_cliente)
REFERENCES clients(id_cliente) ON DELETE RESTRICT;
```

```
98 ALTER TABLE bookings
99 DROP CONSTRAINT bookings_id_cliente_fkey,
100 ADD CONSTRAINT bookings_id_cliente_fkey FOREIGN KEY (id_cliente) REFERENCES clients(id_cliente) ON DELETE RESTRICT;
101
```

Data Output [Messages](#) Notifications

ALTER TABLE

Query returned successfully in 37 msec.

Tabla 3 — Nombre: Review

Tabla	Campo	Tipo	NOT NULL	CHECK	DEFAULT	FK / ON DELETE
review	id_review	SERIAL	✓	—	—	PK — sin FK
review	id_booking	INT	✓	—	—	FK Booking — CASCADE
review	created_date	TIMESTAMP	✓	—	✓	—
review	comment	TEXT	X	—	—	—
review	rating	DECIMAL(3,2)	✓	✓	—	—

ALTER TABLE para agregar restricciones faltantes (Tabla 3):

```
ALTER TABLE reviews
  ALTER COLUMN created_date TYPE TIMESTAMP,
  ALTER COLUMN comment TYPE TEXT,
  DROP CONSTRAINT reviews_id_booking_fkey,
  ADD CONSTRAINT reviews_id_booking_fkey FOREIGN KEY (id_booking)
REFERENCES bookings(id_booking) ON DELETE CASCADE;
```

```
102 ALTER TABLE reviews
103
104     ALTER COLUMN created_date TYPE TIMESTAMP,
105
106     ALTER COLUMN comment TYPE TEXT,
107
108     DROP CONSTRAINT reviews_id_booking_fkey,
109
110     ADD CONSTRAINT reviews_id_booking_fkey FOREIGN KEY (id_booking) REFERENCES bookings(id_booking) ON DELETE CASCADE;
```

Data Output Messages Notifications

ALTER TABLE

Query returned successfully in 38 msec.

Tabla 4 — Nombre: Photo_reference

Tabla	Campo	Tipo	NOT NULL	CHECK	DEFAULT	FK / ON DELETE
photo_ref	id_photo_ref	SERIAL	✓	—	—	PK — sin FK
photo_ref	id_review	INT	✓	—	—	FK Review — CASCADE
photo_ref	photo	VARCHAR(255)	✓	—	—	—
photo_ref	s3_bucket_url	VARCHAR(255)	✓	—	—	—

ALTER TABLE para agregar restricciones faltantes (Tabla 4):

```
ALTER TABLE photo_reference
  ALTER COLUMN photo SET NOT NULL,
  DROP CONSTRAINT photo_reference_id_review_fkey,
  ADD CONSTRAINT photo_reference_id_review_fkey FOREIGN KEY (id_review)
REFERENCES reviews(id_review) ON DELETE CASCADE;
```

```
113 ALTER TABLE photo_reference
114
115 ALTER COLUMN photo SET NOT NULL,
116
117 DROP CONSTRAINT photo_reference_id_review_fkey,
118
119 ADD CONSTRAINT photo_reference_id_review_fkey FOREIGN KEY (id_review) REFERENCES reviews(id_review) ON DELETE CASCADE;
```

Data Output Messages Notifications

ALTER TABLE

Query returned successfully in 30 msec.

Tabla 5 — Nombre: Brand

Tabla	Campo	Tipo	NOT NULL	CHECK	DEFAULT	FK / ON DELETE
brand	id_brand	SERIAL	✓	—	—	PK — sin FK
brand	brand_name	VARCHAR(100)	✓	—	—	—
brand	logo_url	VARCHAR(255)	X	—	—	—
brand	description	TEXT	X	—	—	—

ALTER TABLE para agregar restricciones faltantes (Tabla 5):

```
ALTER TABLE brands
  ALTER COLUMN description TYPE TEXT;
```

```
121 ALTER TABLE brands
122
123 ALTER COLUMN description TYPE TEXT;
```

Data Output Messages Notifications

ALTER TABLE

Query returned successfully in 35 msec.

Tabla 6 — Nombre: Professional

Tabla	Campo	Tipo	NOT NULL	CHECK	DEFAULT	FK / ON DELETE
profesional	id_profesional	SERIAL	✓	—	—	PK — sin FK

profesional	id_brand	INT	✗	—	—	FK Brand — SET NULL
profesional	user_name	VARCHAR(50)	✓	—	—	—
profesional	bio_experience	TEXT	✗	—	—	—
profesional	status	VARCHAR(20)	✓	✓	✓	—
profesional	phone	VARCHAR(20)	✓	—	—	—
profesional	rating	DECIMAL(3,2)	✗	✓	—	—
profesional	speciality	VARCHAR(100)	✓	—	—	—

ALTER TABLE para agregar restricciones faltantes (Tabla 6):

```
ALTER TABLE professionals
  ALTER COLUMN id_brand DROP NOT NULL,
  ALTER COLUMN phone SET NOT NULL,
  DROP CONSTRAINT professionals_id_brand_fkey,
  ADD CONSTRAINT professionals_id_brand_fkey FOREIGN KEY (id_brand)
  REFERENCES brands(id_brand) ON DELETE SET NULL;
```

```
124
125 ALTER TABLE professionals
126
127     ALTER COLUMN id_brand DROP NOT NULL,
128
129     ALTER COLUMN phone SET NOT NULL,
130
131     DROP CONSTRAINT professionals_id_brand_fkey,
132
133     ADD CONSTRAINT professionals_id_brand_fkey FOREIGN KEY (id_brand) REFERENCES brands(id_brand) ON DELETE SET NULL;
```

Data Output Messages Notifications

ALTER TABLE

Query returned successfully in 31 msec.

Tabla 7 — Nombre: Service

Tabla	Campo	Tipo	NOT NULL	CHECK	DEFAULT	FK / ON DELETE
service	id_service	SERIAL	✓	—	—	PK — sin FK
service	estimated_dur	INT	✓	✓	—	—
service	description	TEXT	✗	—	—	—

Service	base_price	DECIMAL(10,2)	✓	✓	—	—
---------	------------	---------------	---	---	---	---

ALTER TABLE para agregar restricciones faltantes (Tabla 7):

```
ALTER TABLE services
  ALTER COLUMN description TYPE TEXT;
```

```
135 ALTER TABLE services
```

```
136
```

```
137     ALTER COLUMN description TYPE TEXT;
```

```
138
```

```
139
```

Data Output Messages Notifications

```
ALTER TABLE
```

```
Query returned successfully in 31 msec.
```

Tabla 8 — Nombre: CoverageArea

Tabla	Campo	Tipo	NOT NULL	CHECK	DEFAULT	FK / ON DELETE
coverage	id_coverage	SERIAL	✓	—	—	PK — sin FK
coverage	zip_code	VARCHAR(20)	✓	—	—	—
coverage	neighborhood	VARCHAR(100)	✓	—	—	—

Tabla 9 — Nombre: Categoría

Tabla	Campo	Tipo	NOT NULL	CHECK	DEFAULT	FK / ON DELETE
categoría	id_categoria	SERIAL	✓	—	—	PK — sin FK
categoría	id_service	INT	✓	—	—	FK Service — CASCADE
categoría	nombre	VARCHAR(100)	✓	—	—	—

ALTER TABLE para agregar restricciones faltantes (Tabla 9):

```
ALTER TABLE categorias
  RENAME COLUMN id TO id_categoria;
```

94

95 ▾

```
ALTER TABLE categorias
  RENAME COLUMN id TO id_categoria;
```

96

97

Data Output Messages Notifications

ALTER TABLE

Query returned successfully in 35 msec.

Tabla 10 — Nombre: Professional_Service

Tabla	Campo	Tipo	NOT NULL	CHECK	DEFAULT	FK / ON DELETE
prof_serv	id_profesional	INT	✓	—	—	PK / FK Prof — CASCADE
prof_serv	id_service	INT	✓	—	—	PK / FK Service — CASCADE

Create TABLE para agregar las relaciones segun normalizacion (Tabla 10):

```
CREATE TABLE professional_service (
  id_profesional INTEGER NOT NULL REFERENCES professionals(id_profesional)
  ON DELETE CASCADE,
  id_service INTEGER NOT NULL REFERENCES services(id_service) ON DELETE
  CASCADE,
  PRIMARY KEY (id_profesional, id_service)
);
```

 Adjuntar screenshot del resultado en la consola.

Tabla 11 — Nombre: Professional_Coverage

Tabla	Campo	Tipo	NOT NULL	CHECK	DEFAULT	FK / ON DELETE
-------	-------	------	----------	-------	---------	----------------

prof_cov	id_profesional	INT	✓	—	—	PK / FK Prof — CASCADE
prof_cov	id_coverage	INT	✓	—	—	PK / FK Coverage — CASCADE
Prof_cov	Id_profesional	INT	✓	—	—	PF / FK profesional --- CASCADE

Create TABLE para agregar las relaciones según normalización (Tabla 11):

```
-- Relación M:N entre Professionals y Coverage Areas
CREATE TABLE professional_coverage (
  id_profesional INTEGER NOT NULL REFERENCES professionals(id_profesional)
  ON DELETE CASCADE,
  id_coverage INTEGER NOT NULL REFERENCES coverage_areas(id_coverage) ON
  DELETE CASCADE,
  PRIMARY KEY (id_profesional, id_coverage)
);
```

Tabla 12— Nombre: Booking_Service

Tabla	Campo	Tipo	NOT NULL	CHECK	DEFAULT	FK / ON DELETE
book_serv	id_booking	INT	✓	—	—	PK / FK Booking — CASCADE
book_serv	id_service	INT	✓	—	—	PK / FK Service — CASCADE

Create TABLE para agregar las relaciones según normalización (Tabla 12):

```
-- Relación M:N entre Bookings y Services

CREATE TABLE booking_service (

id_booking INTEGER NOT NULL REFERENCES bookings(id_booking) ON DELETE CASCADE,

id_service INTEGER NOT NULL REFERENCES services(id_service) ON DELETE CASCADE,

PRIMARY KEY (id_booking, id_service)

);
```

PARTE 2 Revisión de FK

20 min — Decidan el comportamiento ON DELETE para cada relación

Referencia rápida:

RESTRICT No deja borrar el padre si tiene hijos activos. Úsalo cuando los hijos son registros importantes (contables, históricos).

CASCADE Borra los hijos automáticamente al borrar el padre. Úsalo cuando el hijo no tiene sentido sin el padre (ej: detalle de orden).

SET NULL Deja la FK del hijo en NULL. Úsalo cuando el hijo puede existir de forma independiente (ej: artículo cuyo autor se elimina).

Instrucciones: Completen una fila por cada FK del proyecto. Justifiquen en lenguaje de negocio por qué eligieron ese comportamiento.

Tabla	Campo FK	→ Referencia	ON DELETE	ON UPDATE	Justificación de negocio
Booking	Id_cliente	Client	RESTRICT	CASCADE	No se debería poder borrar un cliente si tiene reservas activas o no
Review	Id_booking	Booking	CASCADE	CASCADE	Si se borra una reserva no habra review, por lo tanto, no existiria la review
Photo_Reference	Id_review	Review	CASCADE	CASCADE	Si se borra una review no se debe guardar las fotos relacionadas
Profesional	Id_Brand	Brand	SET NULL	CASCADE	Si se elimina un patrocinador es mejor dejar el espacio como null en vez de no dejar
Categoria	Id_service	Servide	Cascade	Cascade	Si se borra una categoria, se deberian borrar todos los servicios asociados a ella

Prof_service	Id_proffesional	profesional	Cascade	Cascade	Si se borra el profesiona no se deberia guardar la relacion de este con el servicio
Prof_service	Id_service	Service	Cascade	Cascade	Si se borra el servicio no se deberia guardar la relacion de este con el profseional
Porf_cov	Id_coverage	Coverage_area	CASCADE	Cascade	Al borrar un area por x motivos, no se deberia guardar la relacion de esta con algun profesional
Prof_cov	Id_professional	Profesional	CASCADE	Cascade	Si se borra a un profesional no se deberia guardar su relacion con las areas donde hacia su trabajo
Booking_service	Id_booking	Booking	Cascade	Cascade	Sis e borra una reserva no se deberia guardar su relacion con el servicio
Bookign_service	Id_service	Service	Cascade	Cascade	Si se borra un servicio no se deberian guardar todas las reservas que tenia asociadas

Script ALTER TABLE para aplicar los ON DELETE elegidos:

Recuerden el proceso en PostgreSQL:

1. DROP CONSTRAINT nombre_actual → 2. ADD CONSTRAINT nuevo_nombre FK ... ON DELETE comportamiento

1. Booking -> Client

No permite borrar un cliente si tiene reservas asociadas.

```
ALTER TABLE bookings
  DROP CONSTRAINT IF EXISTS bookings_id_cliente_fkey,
  ADD CONSTRAINT bookings_id_cliente_fkey
  FOREIGN KEY (id_cliente)
  REFERENCES clients(id_cliente)
  ON DELETE RESTRICT
  ON UPDATE CASCADE;
```

```
ALTER TABLE
```

```
Query returned successfully in 73 msec.
```

2. Review -> Booking

Si se elimina una reserva, también se elimina la reseña asociada.

```
ALTER TABLE reviews
  DROP CONSTRAINT IF EXISTS reviews_id_booking_fkey,
  ADD CONSTRAINT reviews_id_booking_fkey
  FOREIGN KEY (id_booking)
  REFERENCES bookings(id_booking)
  ON DELETE CASCADE
  ON UPDATE CASCADE;
```

```
ALTER TABLE
```

```
Query returned successfully in 89 msec.
```

3. Photo_Reference -> Review

Si se elimina una reseña, también se eliminan sus fotos relacionadas.

```
ALTER TABLE photo_reference
  DROP CONSTRAINT IF EXISTS photo_reference_id_review_fkey,
  ADD CONSTRAINT photo_reference_id_review_fkey
  FOREIGN KEY (id_review)
  REFERENCES reviews(id_review)
  ON DELETE CASCADE
  ON UPDATE CASCADE;
```

```
ALTER TABLE
```

```
Query returned successfully in 52 msec.
```

4. Professional -> Brand

Si se elimina una marca, el profesional puede seguir existiendo sin marca asociada.

```
ALTER TABLE professionals
  ALTER COLUMN id_brand DROP NOT NULL,
  DROP CONSTRAINT IF EXISTS professionals_id_brand_fkey,
  ADD CONSTRAINT professionals_id_brand_fkey
  FOREIGN KEY (id_brand)
  REFERENCES brands(id_brand)
  ON DELETE SET NULL
  ON UPDATE CASCADE;
ALTER TABLE
```

Query returned successfully in 86 msec.

5. Service -> Categoria

No permite borrar una categoría si tiene servicios asociados.

```
ALTER TABLE services
  DROP CONSTRAINT IF EXISTS services_id_categoria_fkey,
  ADD CONSTRAINT services_id_categoria_fkey
  FOREIGN KEY (id_categoria)
  REFERENCES categorias(id_categoria)
  ON DELETE RESTRICT
  ON UPDATE CASCADE;
```

```
ALTER TABLE
```

Query returned successfully in 179 msec.

6. Professional_Service -> Professional

Si se borra el profesional no se guarda su relación con servicios.

```
ALTER TABLE professional_service DROP CONSTRAINT IF EXISTS
professional_service_id_profesional_fkey, ADD CONSTRAINT
professional_service_id_profesional_fkey FOREIGN KEY (id_profesional)
REFERENCES professionals(id_profesional) ON DELETE CASCADE ON UPDATE
CASCADE;
```

```
ALTER TABLE
```

Query returned successfully in 118 msec.

7. Professional_Service -> Service

Si se borra el servicio no se guarda su relación con profesionales.

```
ALTER TABLE professional_service
```

```

DROP CONSTRAINT IF EXISTS professional_service_id_service_fkey,
ADD CONSTRAINT professional_service_id_service_fkey
FOREIGN KEY (id_service)
REFERENCES services(id_service)
ON DELETE CASCADE
ON UPDATE CASCADE;

```

8. Professional_Coverage -> Coverage_Area

Al borrar un área, no se guarda su relación con profesionales

```

ALTER TABLE professional_coverage
DROP CONSTRAINT IF EXISTS professional_coverage_id_coverage_fkey,
ADD CONSTRAINT professional_coverage_id_coverage_fkey
FOREIGN KEY (id_coverage)
REFERENCES coverage_areas(id_coverage)
ON DELETE CASCADE
ON UPDATE CASCADE;

```

9. Professional_Coverage -> Professional

Si se borra un profesional no se guarda su relación con áreas de cobertura.

```

ALTER TABLE professional_coverage
DROP CONSTRAINT IF EXISTS professional_coverage_id_profesional_fkey,
ADD CONSTRAINT professional_coverage_id_profesional_fkey
FOREIGN KEY (id_profesional)
REFERENCES professionals(id_profesional)
ON DELETE CASCADE
ON UPDATE CASCADE;

```

```
ALTER TABLE
```

```
Query returned successfully in 89 msec.
```

10. Booking_Service -> Booking

Si se borra una reserva no se guarda su relación con servicios.

```
ALTER TABLE booking_service  
DROP CONSTRAINT IF EXISTS booking_service_id_booking_fkey,  
ADD CONSTRAINT booking_service_id_booking_fkey  
FOREIGN KEY (id_booking)  
REFERENCES bookings(id_booking)  
ON DELETE CASCADE  
ON UPDATE CASCADE;
```

```
Data Output Messages Notifications
```

```
ALTER TABLE
```

```
Query returned successfully in 154 msec.
```

11. Booking_Service -> Service

Si se borra un servicio no se guardan las reservas asociadas a él.

```
ALTER TABLE booking_service  
DROP CONSTRAINT IF EXISTS booking_service_id_service_fkey,  
ADD CONSTRAINT booking_service_id_service_fkey  
FOREIGN KEY (id_service)  
REFERENCES services(id_service)  
ON DELETE CASCADE  
ON UPDATE CASCADE;
```

```
Data Output Messages Notifications
```

```
ALTER TABLE
```

```
Query returned successfully in 100 msec.
```

 Adjuntar screenshot del resultado en la consola.

PARTE 3 Pruebas de integridad

25 min — Violen sus restricciones a propósito

¿Por qué hacer esto?

La única forma de saber que una restricción funciona es intentar violarla. Si la BD rechaza el INSERT/DELETE con el error correcto, la restricción está bien configurada. Si no hay error, algo está mal.

Para cada prueba: escriban el SQL que intenta violar la restricción, ejecuten, capturen el error y anótenlo.

Prueba 1 — Violación de CHECK

Intenten insertar un registro que viole UNO de sus CHECK constraints. Por ejemplo, un valor negativo en un campo numérico o una fecha inválida.

¿Qué CHECK están violando?

Se viola la restricción CHECK del campo base_price en la tabla services, porque el precio base no puede ser negativo.

SQL que genera la violación + error recibido:

```
INSERT INTO services (  
    id_categoria,  
    name,  
    description,  
    base_price,  
    estimated_duration  
)  
VALUES (  
    1,  
    'Maquillaje social',  
    'Servicio de maquillaje para evento',  
    -50000,  
    60  
);
```

ERROR: el nuevo registro para la relación «services» viola la restricción «check» «services_base_price_check»
La fila que falla contiene (1, 1, Maquillaje social, Servicio de maquillaje para evento, -50000.00, 60).

SQL state: 23514

 Adjuntar screenshot del resultado en la consola.

 **Screenshot del error en consola obligatorio para que cuente como entregado.**

Prueba 2 — Violación de FK con RESTRICT

Intenten eliminar (DELETE) un registro padre que tenga hijos activos en la BD, donde la FK usa RESTRICT. Por ejemplo, borrar una categoría que tiene productos, o un cliente que tiene pedidos.

¿Qué relación están probando? (padre → hijo):

Se prueba la relación clients → bookings, donde bookings.id_cliente depende de clients.id_cliente. Como la FK usa ON DELETE RESTRICT, no se debe permitir eliminar un cliente que tenga reservas asociadas.

SQL que genera la violación + error recibido:

```
Se creo un cliente previamente y su id es 1
DELETE FROM clients
WHERE id_cliente = 1;
```

ERROR: update o delete en «clients» viola la llave foránea «bookings_id_cliente_fkey» en la tabla «bookings»
La llave (id_cliente)=(1) todavía es referida desde la tabla «bookings».

SQL state: 23503

 Adjuntar screenshot del resultado en la consola.

 **Screenshot del error en consola obligatorio.**

Prueba 3 — Violación de NOT NULL

Intenten insertar un registro omitiendo un campo que tiene NOT NULL. El INSERT debe especificar solo algunos campos y dejar fuera el que tiene la restricción.

¿Qué campo NOT NULL están omitiendo?:

Se omite el campo phone de la tabla clients, el cual fue definido como NOT NULL.

SQL que genera la violación + error recibido:

```
INSERT INTO clients (first_name, last_name, email, password_hash,
address) VALUES
('Valentina', 'Ríos', 'vale.rios@email.com', 'hash_abc123', 'Cra
7 # 45-10, Chapinero');
```

Data Output Messages Notifications

ERROR: el valor nulo en la columna «phone» de la relación «clients» viola la restricción de no nulo
La fila que falla contiene (1, Valentina, Ríos, vale.rios@email.com, null, hash_abc123, Cra 7 # 45-10,
Chapinero).

SQL state: 23502

Detail: La fila que falla contiene (1, Valentina, Ríos, vale.rios@email.com, null, hash_abc123, Cra 7 #
45-10, Chapinero).

 Adjuntar screenshot del resultado en la consola.

 **Screenshot del error en consola obligatorio.**

Reflexión de cierre

Discutan y anoten brevemente:

¿Alguna de las pruebas de integridad falló de una forma inesperada? ¿Descubrieron alguna restricción que faltaba en su proyecto y que no habían pensado antes?

Durante las pruebas de integridad se comprobó que las restricciones definidas cumplen una función importante para proteger la consistencia de la base de datos. La violación del CHECK permitió verificar que el sistema no acepta valores incoherentes, como precios negativos en los servicios. La prueba de FK con RESTRICT evidenció que no es posible eliminar un cliente que ya tiene reservas asociadas, lo cual conserva el historial operativo del negocio. Finalmente, la prueba de NOT NULL confirmó que los campos obligatorios no pueden quedar vacíos, garantizando que la información mínima necesaria sea registrada correctamente.

También se identificó la importancia de declarar explícitamente los comportamientos ON DELETE y ON UPDATE, ya que no basta con crear las llaves foráneas: cada relación debe responder a una lógica de negocio. En este caso, se aplicó RESTRICT para proteger registros históricos, CASCADE para eliminar datos dependientes que no tienen sentido por sí solos, y SET NULL cuando la entidad hija puede seguir existiendo sin la entidad padre.

☒ Checklist de entrega — DDL del proyecto (Semana 3)

Verifiquen que su script DDL final incluye:

- ☐ CREATE TABLE con restricciones: NOT NULL, UNIQUE, CHECK y DEFAULT donde aplica.
- ☐ Todas las FK con ON DELETE y ON UPDATE explícitos (no confiar en el default NO ACTION).

- ☐ Nombres explícitos en todos los CONSTRAINT (no nombres automáticos).
- ☐ Script ejecutable desde cero: DROP IF EXISTS en orden → CREATE en orden de dependencia.
- ☐ Screenshots de las 3 pruebas de integridad (CHECK, FK RESTRICT, NOT NULL).